

We see a strong pattern in the numbers of Saturday flights. This is reassuring, because we also saw that pattern in the raw data. It's a good sign when you get the same signal from different approaches.

Exercises

1. Use your Google sleuthing skills to brainstorm why there were fewer than expected flights on January 20, May 26, and September 1. (Hint: they all have the same explanation.) How would these days generalize to another year?
2. What do the three days with high positive residuals represent? How would these days generalize to another year?

```
daily %>%  
  top_n(3, resid)  
#> # A tibble: 3 × 5  
#>   date       n wday resid term  
#>   <date> <int> <ord> <dbl> <fctr>  
#> 1 2013-11-30  857  Sat  112.4 fall  
#> 2 2013-12-01  987  Sun   95.5 fall  
#> 3 2013-12-28  814  Sat   69.4 fall
```

3. Create a new variable that splits the `wday` variable into terms, but only for Saturdays, i.e., it should have Thurs, Fri, but Sat-summer, Sat-spring, Sat-fall. How does this model compare with the model with every combination of `wday` and `term`?
4. Create a new `wday` variable that combines the day of week, term (for Saturdays), and public holidays. What do the residuals of that model look like?
5. What happens if you fit a day-of-week effect that varies by month (i.e., `n ~ wday * month`)? Why is this not very helpful?
6. What would you expect the model `n ~ wday + ns(date, 5)` to look like? Knowing what you know about the data, why would you expect it to be not particularly effective?
7. We hypothesized that people leaving on Sundays are more likely to be business travelers who need to be somewhere on Monday. Explore that hypothesis by seeing how it breaks down based on distance and time: if it's true, you'd expect to see more Sunday evening flights to places that are far away.

8. It's a little frustrating that Sunday and Saturday are on separate ends of the plot. Write a small function to set the levels of the factor so that the week starts on Monday.

Learning More About Models

We have only scratched the absolute surface of modeling, but you have hopefully gained some simple, but general-purpose tools that you can use to improve your own data analyses. It's OK to start simple! As you've seen, even very simple models can make a dramatic difference in your ability to tease out interactions between variables.

These modeling chapters are even more opinionated than the rest of the book. I approach modeling from a somewhat different perspective to most others, and there is relatively little space devoted to it. Modeling really deserves a book on its own, so I'd highly recommend that you read at least one of these three books:

- *Statistical Modeling: A Fresh Approach* by Danny Kaplan. This book provides a gentle introduction to modeling, where you build your intuition, mathematical tools, and R skills in parallel. The book replaces a traditional “introduction to statistics” course, providing a curriculum that is up-to-date and relevant to data science.
- *An Introduction to Statistical Learning* by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani (available online for free). This book presents a family of modern modeling techniques collectively known as statistical learning. For an even deeper understanding of the math behind the models, read the classic *Elements of Statistical Learning* by Trevor Hastie, Robert Tibshirani, and Jerome Friedman (also available online for free).
- *Applied Predictive Modeling* by Max Kuhn and Kjell Johnson. This book is a companion to the **caret** package and provides practical tools for dealing with real-life predictive modeling challenges.

Many Models with purrr and broom

Introduction

In this chapter you're going to learn three powerful ideas that help you to work with large numbers of models with ease:

- Using many simple models to better understand complex datasets.
- Using list-columns to store arbitrary data structures in a data frame. For example, this will allow you to have a column that contains linear models.
- Using the **broom** package, by David Robinson, to turn models into tidy data. This is a powerful technique for working with large numbers of models because once you have tidy data, you can apply all of the techniques that you've learned about earlier in the book.

We'll start by diving into a motivating example using data about life expectancy around the world. It's a small dataset but it illustrates how important modeling can be for improving your visualizations. We'll use a large number of simple models to partition out some of the strongest signals so we can see the subtler signals that remain. We'll also see how model summaries can help us pick out outliers and unusual trends.

The following sections will dive into more detail about the individual techniques:

- In “[gapminder](#)” on page 398, you’ll see a motivating example that puts list-columns to use to fit per-county models to world economic data.
- In “[List-Columns](#)” on page 402, you’ll learn more about the list-column data structure, and why it’s valid to put lists in data frames.
- In “[Creating List-Columns](#)” on page 411, you’ll learn the three main ways in which you’ll create list-columns.
- In “[Simplifying List-Columns](#)” on page 416 you’ll learn how to convert list-columns back to regular atomic vectors (or sets of atomic vectors) so you can work with them more easily.
- In “[Making Tidy Data with broom](#)” on page 419, you’ll learn about the full set of tools provided by broom, and see how they can be applied to other types of data structure.

This chapter is somewhat aspirational: if this book is your first introduction to R, this chapter is likely to be a struggle. It requires you to have deeply internalized ideas about modeling, data structures, and iteration. So don’t worry if you don’t get it—just put this chapter aside for a few months, and come back when you want to stretch your brain.

Prerequisites

Working with many models requires many of the packages of the tidyverse (for data exploration, wrangling, and programming) and **modelr** to facilitate modeling.

```
library(modelr)
library(tidyverse)
```

gapminder

To motivate the power of many simple models, we’re going to look into the “gapminder” data. This data was popularized by Hans Rosling, a Swedish doctor and statistician. If you’ve never heard of him, stop reading this chapter right now and go watch one of his videos! He is a fantastic data presenter and illustrates how you can use data

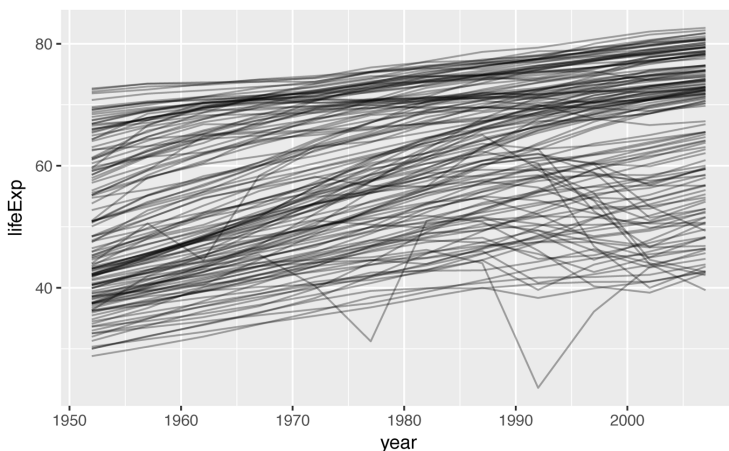
to present a compelling story. A good place to start is this [short video](#) filmed in conjunction with the BBC.

The gapminder data summarizes the progression of countries over time, looking at statistics like life expectancy and GDP. The data is easy to access in R, thanks to Jenny Bryan, who created the **gapminder** package:

```
library(gapminder)
gapminder
#> # A tibble: 1,704 × 6
#>   country continent year lifeExp      pop gdpPercap
#>   <fctr>    <fctr> <int>   <dbl>   <int>    <dbl>
#> 1 Afghanistan Asia  1952    28.8  8425333    779
#> 2 Afghanistan Asia  1957    30.3  9240934    821
#> 3 Afghanistan Asia  1962    32.0 10267083    853
#> 4 Afghanistan Asia  1967    34.0 11537966    836
#> 5 Afghanistan Asia  1972    36.1 13079460    740
#> 6 Afghanistan Asia  1977    38.4 14880372    786
#> # ... with 1,698 more rows
```

In this case study, we’re going to focus on just three variables to answer the question “How does life expectancy (`lifeExp`) change over time (`year`) for each country (`country`)?” A good place to start is with a plot:

```
gapminder %>%
  ggplot(aes(year, lifeExp, group = country)) +
  geom_line(alpha = 1/3)
```



This is a small dataset: it only has ~1,700 observations and 3 variables. But it’s still hard to see what’s going on! Overall, it looks like

life expectancy has been steadily improving. However, if you look closely, you might notice some countries that don't follow this pattern. How can we make those countries easier to see?

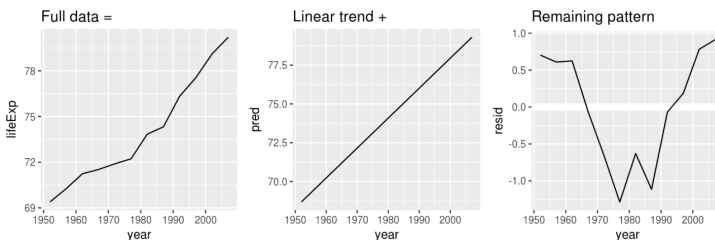
One way is to use the same approach as in the last chapter: there's a strong signal (overall linear growth) that makes it hard to see subtler trends. We'll tease these factors apart by fitting a model with a linear trend. The model captures steady growth over time, and the residuals will show what's left.

You already know how to do that if we had a single country:

```
nz <- filter(gapminder, country == "New Zealand")
nz %>%
  ggplot(aes(year, lifeExp)) +
  geom_line() +
  ggtitle("Full data = ")

nz_mod <- lm(lifeExp ~ year, data = nz)
nz %>%
  add_predictions(nz_mod) %>%
  ggplot(aes(year, pred)) +
  geom_line() +
  ggtitle("Linear trend + ")

nz %>%
  add_residuals(nz_mod) %>%
  ggplot(aes(year, resid)) +
  geom_hline(yintercept = 0, color = "white", size = 3) +
  geom_line() +
  ggtitle("Remaining pattern")
```



How can we easily fit that model to every country?

Nested Data

You could imagine copying and pasting that code multiple times; but you've already learned a better way! Extract out the common

code with a function and repeat using a map function from **purrr**. This problem is structured a little differently to what you've seen before. Instead of repeating an action for each variable, we want to repeat an action for each country, a subset of rows. To do that, we need a new data structure: the *nested data frame*. To create a nested data frame we start with a grouped data frame, and “nest” it:

```
by_country <- gapminder %>%
  group_by(country, continent) %>%
  nest()

by_country
#> # A tibble: 142 × 3
#>   country continent data
#>   <fctr>    <fctr>   <list>
#> 1 Afghanistan Asia <tibble [12 × 4]>
#> 2 Albania Europe <tibble [12 × 4]>
#> 3 Algeria Africa <tibble [12 × 4]>
#> 4 Angola Africa <tibble [12 × 4]>
#> 5 Argentina Americas <tibble [12 × 4]>
#> 6 Australia Oceania <tibble [12 × 4]>
#> # ... with 136 more rows
```

(I'm cheating a little by grouping on both continent and country. Given country, continent is fixed, so this doesn't add any more groups, but it's an easy way to carry an extra variable along for the ride.)

This creates a data frame that has one row per group (per country), and a rather unusual column: `data`. `data` is a list of data frames (or tibbles, to be precise). This seems like a crazy idea: we have a data frame with a column that is a list of other data frames! I'll explain shortly why I think this is a good idea.

The data column is a little tricky to look at because it's a moderately complicated list, and we're still working on good tools to explore these objects. Unfortunately using `str()` is not recommended as it will often produce very long output. But if you pluck out a single element from the `data` column you'll see that it contains all the data for that country (in this case, Afghanistan):

```
by_country$data[[1]]
#> # A tibble: 12 × 4
#>   year lifeExp      pop gdpPercap
#>   <int>   <dbl>   <int>    <dbl>
#> 1 1952   28.8  8425333     779
#> 2 1957   30.3  9240934     821
#> 3 1962   32.0 10267083     853
```

```
#> 4 1967 34.0 11537966 836
#> 5 1972 36.1 13079460 740
#> 6 1977 38.4 14880372 786
#> # ... with 6 more rows
```

Note the difference between a standard grouped data frame and a nested data frame: in a grouped data frame, each row is an observation; in a nested data frame, each row is a group. Another way to think about a nested dataset is we now have a meta-observation: a row that represents the complete time course for a country, rather than a single point in time.

List-Columns

Now that we have our nested data frame, we're in a good position to fit some models. We have a model-fitting function:

```
country_model <- function(df) {
  lm(lifeExp ~ year, data = df)
}
```

And we want to apply it to every data frame. The data frames are in a list, so we can use `purrr::map()` to apply `country_model` to each element:

```
models <- map(by_country$data, country_model)
```

However, rather than leaving the list of models as a free-floating object, I think it's better to store it as a column in the `by_country` data frame. Storing related objects in columns is a key part of the value of data frames, and why I think list-columns are such a good idea. In the course of working with these countries, we are going to have lots of lists where we have one element per country. So why not store them all together in one data frame?

In other words, instead of creating a new object in the global environment, we're going to create a new variable in the `by_country` data frame. That's a job for `dplyr::mutate()`:

```
by_country <- by_country %>%
  mutate(model = map(data, country_model))
by_country
#> # A tibble: 142 × 4
#>   country continent      data      model
#>   <fctr>    <fctr>    <list>   <list>
#> 1 Afghanistan Asia <tibble [12 × 4]> <S3: lm>
#> 2 Albania Europe <tibble [12 × 4]> <S3: lm>
#> 3 Algeria Africa <tibble [12 × 4]> <S3: lm>
```



```
#> 4      Angola      Africa <tibble [12 × 4]> <S3: lm>
#> 5      Argentina Americas <tibble [12 × 4]> <S3: lm>
#> 6      Australia Oceania <tibble [12 × 4]> <S3: lm>
#> # ... with 136 more rows
```

This has a big advantage: because all the related objects are stored together, you don't need to manually keep them in sync when you filter or arrange. The semantics of the data frame takes care of that for you:

```
by_country %>%
  filter(continent == "Europe")
#> # A tibble: 30 × 4
#>       country continent      data      model
#>   <fctr>      <fctr>      <list>    <list>
#> 1      Albania Europe <tibble [12 × 4]> <S3: lm>
#> 2      Austria Europe <tibble [12 × 4]> <S3: lm>
#> 3      Belgium Europe <tibble [12 × 4]> <S3: lm>
#> 4 Bosnia and Herzegovina Europe <tibble [12 × 4]> <S3: lm>
#> 5      Bulgaria Europe <tibble [12 × 4]> <S3: lm>
#> 6      Croatia Europe <tibble [12 × 4]> <S3: lm>
#> # ... with 24 more rows
by_country %>%
  arrange(continent, country)
#> # A tibble: 142 × 4
#>       country continent      data      model
#>   <fctr>      <fctr>      <list>    <list>
#> 1      Algeria Africa <tibble [12 × 4]> <S3: lm>
#> 2      Angola  Africa <tibble [12 × 4]> <S3: lm>
#> 3      Benin   Africa <tibble [12 × 4]> <S3: lm>
#> 4      Botswana Africa <tibble [12 × 4]> <S3: lm>
#> 5 Burkina Faso Africa <tibble [12 × 4]> <S3: lm>
#> 6      Burundi Africa <tibble [12 × 4]> <S3: lm>
#> # ... with 136 more rows
```

If your list of data frames and list of models were separate objects, you have to remember that whenever you reorder or subset one vector, you need to reorder or subset all the others in order to keep them in sync. If you forget, your code will continue to work, but it will give the wrong answer!

Unnesting

Previously we computed the residuals of a single model with a single dataset. Now we have 142 data frames and 142 models. To compute the residuals, we need to call `add_residuals()` with each model-data pair:

```

by_country <- by_country %>%
  mutate(
    resid = map2(data, model, add_residuals)
  )
by_country
#> # A tibble: 142 × 5
#>   country continent data model
#>   <fctr>    <fctr>    <list>  <list>
#> 1 Afghanistan Asia <tibble [12 × 4]> <S3: lm>
#> 2 Albania Europe <tibble [12 × 4]> <S3: lm>
#> 3 Algeria Africa <tibble [12 × 4]> <S3: lm>
#> 4 Angola Africa <tibble [12 × 4]> <S3: lm>
#> 5 Argentina Americas <tibble [12 × 4]> <S3: lm>
#> 6 Australia Oceania <tibble [12 × 4]> <S3: lm>
#> # ... with 136 more rows, and 1 more variable:
#> #   resid <list>

```

But how can you plot a list of data frames? Instead of struggling to answer that question, let's turn the list of data frames back into a regular data frame. Previously we used `nest()` to turn a regular data frame into a nested data frame, and now we do the opposite with `unnest()`:

```

resids <- unnest(by_country, resid)
resids
#> # A tibble: 1,704 × 7
#>   country continent year lifeExp pop gdpPercap
#>   <fctr>    <fctr> <int>   <dbl>   <int>    <dbl>
#> 1 Afghanistan Asia 1952   28.8  8425333    779
#> 2 Afghanistan Asia 1957   30.3  9240934    821
#> 3 Afghanistan Asia 1962   32.0 10267083    853
#> 4 Afghanistan Asia 1967   34.0 11537966    836
#> 5 Afghanistan Asia 1972   36.1 13079460    740
#> 6 Afghanistan Asia 1977   38.4 14880372    786
#> # ... with 1,698 more rows, and 1 more variable: resid <dbl>

```

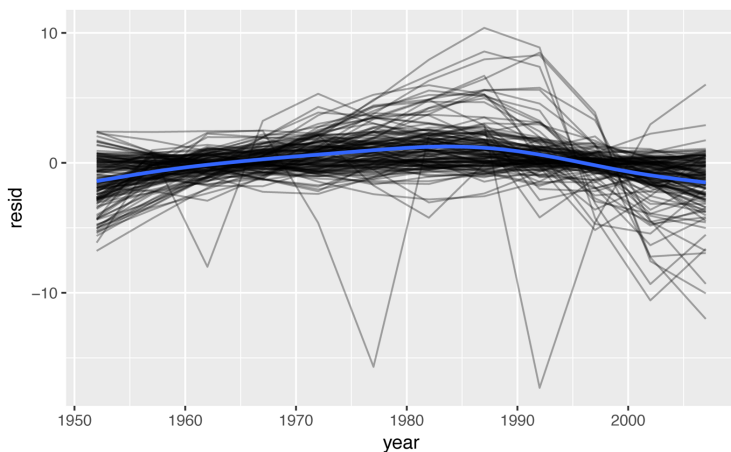
Note that each regular column is repeated once for each row in the nested column.

Now that we have regular data frame, we can plot the residuals:

```

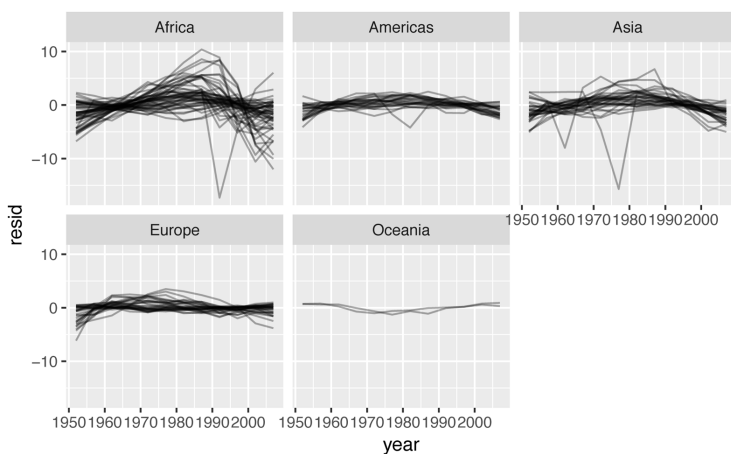
resids %>%
  ggplot(aes(year, resid)) +
  geom_line(aes(group = country), alpha = 1 / 3) +
  geom_smooth(se = FALSE)
#> `geom_smooth()` using method = 'gam'

```



Faceting by continent is particularly revealing:

```
resids %>%
  ggplot(aes(year, resid, group = country)) +
  geom_line(alpha = 1 / 3) +
  facet_wrap(~continent)
```



It looks like we've missed some mild pattern. There's also something interesting going on in Africa: we see some very large residuals, which suggests our model isn't fitting so well there. We'll explore that more in the next section, attacking it from a slightly different angle.

Model Quality

Instead of looking at the residuals from the model, we could look at some general measurements of model quality. You learned how to compute some specific measures in the previous chapter. Here we'll show a different approach using the **broom** package. The **broom** package provides a general set of functions to turn models into tidy data. Here we'll use `broom::glance()` to extract some model quality metrics. If we apply it to a model, we get a data frame with a single row:

```
broom::glance(nz_mod)
#>   r.squared adj.r.squared sigma statistic  p.value df logLik
#>   AIC      BIC
#> 1    0.954          0.949 0.804        205 5.41e-08  2  -13.3
#> 32.6 34.1
#>   deviance df.residual
#> 1    6.47          10
```

We can use `mutate()` and `unnest()` to create a data frame with a row for each country:

```
by_country %>%
  mutate(glance = map(model, broom::glance)) %>%
  unnest(glance)
#> # A tibble: 142 × 16
#>   country continent      data      model
#>   <fctr>    <fctr>    <list>    <list>
#> 1 Afghanistan Asia <tibble [12 × 4]> <S3: lm>
#> 2 Albania Europe <tibble [12 × 4]> <S3: lm>
#> 3 Algeria Africa <tibble [12 × 4]> <S3: lm>
#> 4 Angola Africa <tibble [12 × 4]> <S3: lm>
#> 5 Argentina Americas <tibble [12 × 4]> <S3: lm>
#> 6 Australia Oceania <tibble [12 × 4]> <S3: lm>
#> # ... with 136 more rows, and 12 more variables:
#> #   resid <list>, r.squared <dbl>, adj.r.squared <dbl>,
#> #   sigma <dbl>, statistic <dbl>, p.value <dbl>, df <int>,
#> #   logLik <dbl>, AIC <dbl>, BIC <dbl>, deviance <dbl>,
#> #   df.residual <int>
```

This isn't quite the output we want, because it still includes all the list-columns. This is default behavior when `unnest()` works on single-row data frames. To suppress these columns we use `.drop = TRUE`:

```
glance <- by_country %>%
  mutate(glance = map(model, broom::glance)) %>%
  unnest(glance, .drop = TRUE)
glance
```

```
#> # A tibble: 142 × 13
#>   country continent r.squared adj.r.squared sigma
#>   <fctr>    <fctr>    <dbl>      <dbl> <dbl>
#> 1 Afghanistan Asia      0.948      0.942 1.223
#> 2 Albania Europe    0.911      0.902 1.983
#> 3 Algeria Africa    0.985      0.984 1.323
#> 4 Angola Africa    0.888      0.877 1.407
#> 5 Argentina Americas 0.996      0.995 0.292
#> 6 Australia Oceania   0.980      0.978 0.621
#> # ... with 136 more rows, and 8 more variables:
#> #   statistic <dbl>, p.value <dbl>, df <int>, logLik <dbl>,
#> #   AIC <dbl>, BIC <dbl>, deviance <dbl>, df.residual <int>
```

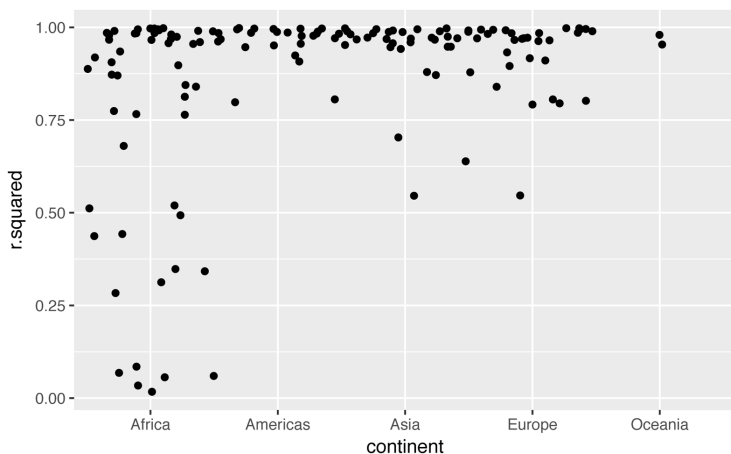
(Pay attention to the variables that aren't printed: there's a lot of useful stuff there.)

With this data frame in hand, we can start to look for models that don't fit well:

```
glance %>%
  arrange(r.squared)
#> # A tibble: 142 × 13
#>   country continent r.squared adj.r.squared sigma
#>   <fctr>    <fctr>    <dbl>      <dbl> <dbl>
#> 1 Rwanda Africa    0.0172    -0.08112 6.56
#> 2 Botswana Africa    0.0340    -0.06257 6.11
#> 3 Zimbabwe Africa    0.0562    -0.03814 7.21
#> 4 Zambia Africa    0.0598    -0.03418 4.53
#> 5 Swaziland Africa    0.0682    -0.02497 6.64
#> 6 Lesotho Africa    0.0849    -0.00666 5.93
#> # ... with 136 more rows, and 8 more variables:
#> #   statistic <dbl>, p.value <dbl>, df <int>, logLik <dbl>,
#> #   AIC <dbl>, BIC <dbl>, deviance <dbl>, df.residual <int>
```

The worst models all appear to be in Africa. Let's double-check that with a plot. Here we have a relatively small number of observations and a discrete variable, so `geom_jitter()` is effective:

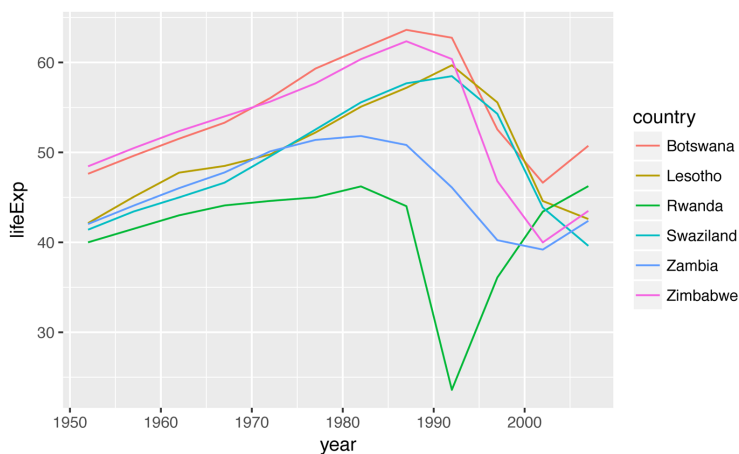
```
glance %>%
  ggplot(aes(continent, r.squared)) +
  geom_jitter(width = 0.5)
```



We could pull out the countries with particularly bad R^2 and plot the data:

```
bad_fit <- filter(glance, r.squared < 0.25)

gapminder %>%
  semi_join(bad_fit, by = "country") %>%
  ggplot(aes(year, lifeExp, color = country)) +
  geom_line()
```



We see two main effects here: the tragedies of the HIV/AIDS epidemic and the Rwandan genocide.

Exercises

1. A linear trend seems to be slightly too simple for the overall trend. Can you do better with a quadratic polynomial? How can you interpret the coefficients of the quadratic? (Hint: you might want to transform `year` so that it has mean zero.)
2. Explore other methods for visualizing the distribution of R^2 per continent. You might want to try the **ggbeeswarm** package, which provides similar methods for avoiding overlaps as jitter, but uses deterministic methods.
3. To create the last plot (showing the data for the countries with the worst model fits), we needed two steps: we created a data frame with one row per country and then semi-joined it to the original dataset. It's possible to avoid this join if we use `unnest()` instead of `unnest(.drop = TRUE)`. How?

List-Columns

Now that you've seen a basic workflow for managing many models, let's dive back into some of the details. In this section, we'll explore the list-column data structure in a little more detail. It's only recently that I've really appreciated the idea of the list-column. List-columns are implicit in the definition of the data frame: a data frame is a named list of equal length vectors. A list is a vector, so it's always been legitimate to use a list as a column of a data frame. However, base R doesn't make it easy to create list-columns, and `data.frame()` treats a list as a list of columns:

```
data.frame(x = list(1:3, 3:5))
#>   x.1.3 x.3.5
#> 1     1     3
#> 2     2     4
#> 3     3     5
```

You can prevent `data.frame()` from doing this with `I()`, but the result doesn't print particularly well:

```
data.frame(
  x = I(list(1:3, 3:5)),
  y = c("1, 2", "3, 4, 5")
)
#>           x           y
```

```
#> 1 1, 2, 3      1, 2
#> 2 3, 4, 5 3, 4, 5
```

Tibble alleviates this problem by being lazier (`tibble()` doesn't modify its inputs) and by providing a better print method:

```
tibble(
  x = list(1:3, 3:5),
  y = c("1, 2", "3, 4, 5")
)
#> # A tibble: 2 × 2
#>       x      y
#>   <list> <chr>
#> 1 <int [3]>    1, 2
#> 2 <int [3]>    3, 4, 5
```

It's even easier with `tribble()` as it can automatically work out that you need a list:

```
tribble(
  ~x, ~y,
  1:3, "1, 2",
  3:5, "3, 4, 5"
)
#> # A tibble: 2 × 2
#>       x      y
#>   <list> <chr>
#> 1 <int [3]>    1, 2
#> 2 <int [3]>    3, 4, 5
```

List-columns are often most useful as an intermediate data structure. They're hard to work with directly, because most R functions work with atomic vectors or data frames, but the advantage of keeping related items together in a data frame is worth a little hassle.

Generally there are three parts of an effective list-column pipeline:

1. You create the list-column using one of `nest()`, `summarize()` + `list()`, or `mutate()` + a map function, as described in [“Creating List-Columns” on page 411](#).
2. You create other intermediate list-columns by transforming existing list columns with `map()`, `map2()`, or `pmap()`. For example, in the previous case study, we created a list-column of models by transforming a list-column of data frames.
3. You simplify the list-column back down to a data frame or atomic vector, as described in [“Simplifying List-Columns” on page 416](#).

Creating List-Columns

Typically, you won't create list-columns with `tibble()`. Instead, you'll create them from regular columns, using one of three methods:

1. With `tidyr::nest()` to convert a grouped data frame into a nested data frame where you have list-column of data frames.
2. With `mutate()` and vectorized functions that return a list.
3. With `summarize()` and summary functions that return multiple results.

Alternatively, you might create them from a named list, using `tibble::enframe()`.

Generally, when creating list-columns, you should make sure they're homogeneous: each element should contain the same type of thing. There are no checks to make sure this is true, but if you use **purrr** and remember what you've learned about type-stable functions, you should find it happens naturally.

With Nesting

`nest()` creates a nested data frame, which is a data frame with a list-column of data frames. In a nested data frame each row is a meta-observation: the other columns give variables that define the observation (like country and continent earlier), and the list-column of data frames gives the individual observations that make up the meta-observation.

There are two ways to use `nest()`. So far you've seen how to use it with a grouped data frame. When applied to a grouped data frame, `nest()` keeps the grouping columns as is, and bundles everything else into the list-column:

```
gapminder %>%
  group_by(country, continent) %>%
  nest()
#> # A tibble: 142 × 3
#>   country continent      data
#>   <fctr>    <fctr>    <list>
#> 1 Afghanistan   Asia <tibble [12 × 4]>
#> 2  Albania     Europe <tibble [12 × 4]>
#> 3  Algeria     Africa <tibble [12 × 4]>
```

```
#> 4      Angola      Africa <tibble [12 × 4]>
#> 5  Argentina Americas <tibble [12 × 4]>
#> 6  Australia Oceania <tibble [12 × 4]>
#> # ... with 136 more rows
```

You can also use it on an ungrouped data frame, specifying which columns you want to nest:

```
gapminder %>%
  nest(year:gdpPercap)
#> # A tibble: 142 × 3
#>   country continent      data
#>   <fctr>      <fctr>      <list>
#> 1 Afghanistan   Asia <tibble [12 × 4]>
#> 2  Albania      Europe <tibble [12 × 4]>
#> 3  Algeria      Africa <tibble [12 × 4]>
#> 4    Angola      Africa <tibble [12 × 4]>
#> 5  Argentina Americas <tibble [12 × 4]>
#> 6  Australia Oceania <tibble [12 × 4]>
#> # ... with 136 more rows
```

From Vectorized Functions

Some useful functions take an atomic vector and return a list. For example, in [Chapter 11](#) you learned about `stringr::str_split()`, which takes a character vector and returns a list of character vectors. If you use that inside `mutate`, you'll get a list-column:

```
df <- tribble(
  ~x1,
  "a,b,c",
  "d,e,f,g"
)

df %>%
  mutate(x2 = stringr::str_split(x1, ","))
#> # A tibble: 2 × 2
#>   x1      x2
#>   <chr>   <list>
#> 1 a,b,c <chr [3]>
#> 2 d,e,f,g <chr [4]>
```

`unnest()` knows how to handle these lists of vectors:

```
df %>%
  mutate(x2 = stringr::str_split(x1, ",")) %>%
  unnest()
#> # A tibble: 7 × 2
#>   x1      x2
#>   <chr> <chr>
#> 1 a,b,c a
```

```
#> 2 a,b,c b
#> 3 a,b,c c
#> 4 d,e,f,g d
#> 5 d,e,f,g e
#> 6 d,e,f,g f
#> # ... with 1 more rows
```

(If you find yourself using this pattern a lot, make sure to check out `tidyr::separate_rows()`, which is a wrapper around this common pattern).

Another example of this pattern is using the `map()`, `map2()`, `pmap()` functions from **purrr**. For example, we could take the final example from “Invoking Different Functions” on page 334 and rewrite it to use `mutate()`:

```
sim <- tribble(
  ~f,      ~params,
  "runif", list(min = -1, max = 1),
  "rnorm", list(sd = 5),
  "rpois", list(lambda = 10)
)

sim %>%
  mutate(sims = invoke_map(f, params, n = 10))
#> # A tibble: 3 × 3
#>   f      params      sims
#>   <chr>    <list>    <list>
#> 1 runif <list [2]> <dbl [10]>
#> 2 rnorm <list [1]> <dbl [10]>
#> 3 rpois <list [1]> <int [10]>
```

Note that technically `sim` isn’t homogeneous because it contains both double and integer vectors. However, this is unlikely to cause many problems since integers and doubles are both numeric vectors.

From Multivalued Summaries

One restriction of `summarize()` is that it only works with summary functions that return a single value. That means that you can’t use it with functions like `quantile()` that return a vector of arbitrary length:

```
mtcars %>%
  group_by(cyl) %>%
  summarize(q = quantile(mpg))
#> Error in eval(expr, envir, enclos): expecting a single value
```

You can however, wrap the result in a list! This obeys the contract of `summarize()`, because each summary is now a list (a vector) of length 1:

```
mtcars %>%
  group_by(cyl) %>%
  summarize(q = list(quantile(mpg)))
#> # A tibble: 3 × 2
#>   cyl      q
#>   <dbl>   <list>
#> 1     4 <dbl [5]>
#> 2     6 <dbl [5]>
#> 3     8 <dbl [5]>
```

To make useful results with `unnest()`, you'll also need to capture the probabilities:

```
probs <- c(0.01, 0.25, 0.5, 0.75, 0.99)
mtcars %>%
  group_by(cyl) %>%
  summarize(p = list(probs), q = list(quantile(mpg, probs))) %>%
  unnest()
#> # A tibble: 15 × 3
#>   cyl     p     q
#>   <dbl> <dbl> <dbl>
#> 1     4 0.01  21.4
#> 2     4 0.25  22.8
#> 3     4 0.50  26.0
#> 4     4 0.75  30.4
#> 5     4 0.99  33.8
#> 6     6 0.01  17.8
#> # ... with 9 more rows
```

From a Named List

Data frames with list-columns provide a solution to a common problem: what do you do if you want to iterate over both the contents of a list and its elements? Instead of trying to jam everything into one object, it's often easier to make a data frame: one column can contain the elements, and one column can contain the list. An easy way to create such a data frame from a list is `tibble::enframe()`:

```
x <- list(
  a = 1:5,
  b = 3:4,
  c = 5:6
)
```

```
df <- enframe(x)
df
#> # A tibble: 3 × 2
#>   name      value
#>   <chr>    <list>
#> 1     a <int [5]>
#> 2     b <int [2]>
#> 3     c <int [2]>
```

The advantage of this structure is that it generalizes in a straightforward way—names are useful if you have a character vector of meta-data, but don't help if you have other types of data, or multiple vectors.

Now if you want to iterate over names and values in parallel, you can use `map2()`:

```
df %>%
  mutate(
    smry = map2_chr(
      name,
      value,
      ~ stringr::str_c(.x, ":", .y[1])
    )
  )
#> # A tibble: 3 × 3
#>   name      value smry
#>   <chr>    <list> <chr>
#> 1     a <int [5]> a: 1
#> 2     b <int [2]> b: 3
#> 3     c <int [2]> c: 5
```

Exercises

1. List all the functions that you can think of that take an atomic vector and return a list.
2. Brainstorm useful summary functions that, like `quantile()`, return multiple values.
3. What's missing in the following data frame? How does `quantile()` return that missing piece? Why isn't that helpful here?

```
mtcars %>%
  group_by(cyl) %>%
  summarize(q = list(quantile(mpg))) %>%
  unnest()
#> # A tibble: 15 × 2
#>   cyl      q
#>   <dbl> <dbl>
```

```
#> 1      4 21.4
#> 2      4 22.8
#> 3      4 26.0
#> 4      4 30.4
#> 5      4 33.9
#> 6      6 17.8
#> # ... with 9 more rows
```

4. What does this code do? Why might it be useful?

```
mtcars %>%
  group_by(cyl) %>%
  summarize_each(funs(list))
```

Simplifying List-Columns

To apply the techniques of data manipulation and visualization you’ve learned in this book, you’ll need to simplify the list-column back to a regular column (an atomic vector), or set of columns. The technique you’ll use to collapse back down to a simpler structure depends on whether you want a single value per element, or multiple values:

- If you want a single value, use `mutate()` with `map_lgl()`, `map_int()`, `map_dbl()`, and `map_chr()` to create an atomic vector.
- If you want many values, use `unnest()` to convert list-columns back to regular columns, repeating the rows as many times as necessary.

These are described in more detail in the following sections.

List to Vector

If you can reduce your list column to an atomic vector then it will be a regular column. For example, you can always summarize an object with its type and length, so this code will work regardless of what sort of list-column you have:

```
df <- tribble(
  ~x,
  letters[1:5],
  1:3,
  runif(5)
)
```

```
df %>% mutate(
  type = map_chr(x, typeof),
  length = map_int(x, length)
)
#> # A tibble: 3 × 3
#>       x      type length
#>   <list> <chr> <int>
#> 1 <chr [5]> character     5
#> 2 <int [3]> integer       3
#> 3 <dbl [5]> double        5
```

This is the same basic information that you get from the default `tbl` print method, but now you can use it for filtering. This is a useful technique if you have a heterogeneous list, and want to filter out the parts that aren't working for you.

Don't forget about the `map_*()` shortcuts—you can use `map_chr(x, "apple")` to extract the string stored in `apple` for each element of `x`. This is useful for pulling apart nested lists into regular columns. Use the `.null` argument to provide a value to use if the element is missing (instead of returning `NULL`):

```
df <- tribble(
  ~x,
  list(a = 1, b = 2),
  list(a = 2, c = 4)
)
df %>% mutate(
  a = map_dbl(x, "a"),
  b = map_dbl(x, "b", .null = NA_real_)
)
#> # A tibble: 2 × 3
#>       x      a      b
#>   <list> <dbl> <dbl>
#> 1 <list [2]>     1     2
#> 2 <list [2]>     2    NA
```

Unnesting

`unnest()` works by repeating the regular columns once for each element of the list-column. For example, in the following very simple example we repeat the first row four times (because there the first element of `y` has length four), and the second row once:

```
tibble(x = 1:2, y = list(1:4, 1)) %>% unnest(y)
#> # A tibble: 5 × 2
#>       x      y
#>   <int> <dbl>
```

```
#> 1      1      1
#> 2      1      2
#> 3      1      3
#> 4      1      4
#> 5      2      1
```

This means that you can't simultaneously unnest two columns that contain a different number of elements:

```
# Ok, because y and z have the same number of elements in
# every row
df1 <- tribble(
  ~x, ~y, ~z,
  1, c("a", "b"), 1:2,
  2, "c", 3
)
df1
#> # A tibble: 2 × 3
#>       x         y         z
#>   <dbl>   <list>   <list>
#> 1     1 1 <chr [2]> 1 <int [2]>
#> 2     2 2 <chr [1]> 2 <dbl [1]>
df1 %>% unnest(y, z)
#> # A tibble: 3 × 3
#>       x         y         z
#>   <dbl>   <chr>   <dbl>
#> 1     1     a         1
#> 2     1     b         2
#> 3     2     c         3

# Doesn't work because y and z have different number of elements
df2 <- tribble(
  ~x, ~y, ~z,
  1, "a", 1:2,
  2, c("b", "c"), 3
)
df2
#> # A tibble: 2 × 3
#>       x         y         z
#>   <dbl>   <list>   <list>
#> 1     1 1 <chr [1]> 1 <int [2]>
#> 2     2 2 <chr [2]> 2 <dbl [1]>
df2 %>% unnest(y, z)
#> Error: All nested columns must have
#> the same number of elements.
```

The same principle applies when unnesting list-columns of data frames. You can unnest multiple list-columns as long as all the data frames in each row have the same number of rows.

Exercises

1. Why might the `lengths()` function be useful for creating atomic vector columns from list-columns?
2. List the most common types of vector found in a data frame. What makes lists different?

Making Tidy Data with broom

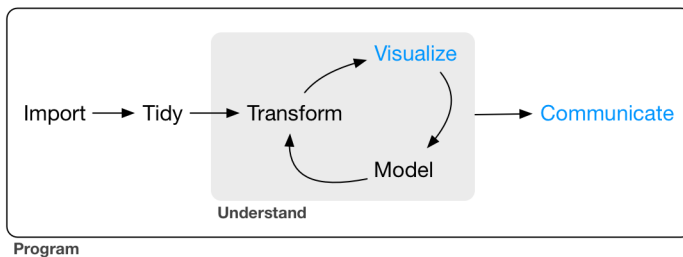
The **broom** package provides three general tools for turning models into tidy data frames:

- `broom::glance(model)` returns a row for each model. Each column gives a model summary: either a measure of model quality, or complexity, or a combination of the two.
- `broom::tidy(model)` returns a row for each coefficient in the model. Each column gives information about the estimate or its variability.
- `broom::augment(model, data)` returns a row for each row in `data`, adding extra values like residuals, and influence statistics.

Broom works with a wide variety of models produced by the most popular modelling packages. See <https://github.com/tidyverse/broom> for a list of currently supported models.

Communicate

So far, you've learned the tools to get your data into R, tidy it into a form convenient for analysis, and then understand your data through transformation, visualization, and modeling. However, it doesn't matter how great your analysis is unless you can explain it to others: you need to *communicate* your results.



Communication is the theme of the following four chapters:

- In [Chapter 21](#), you will learn about R Markdown, a tool for integrating prose, code, and results. You can use R Markdown in notebook mode for analyst-to-analyst communication, and in report mode for analyst-to-decision-maker communication. Thanks to the power of R Markdown formats, you can even use the same document for both purposes.

- In [Chapter 22](#), you will learn how to take your exploratory graphics and turn them into expository graphics, graphics that help the newcomer to your analysis understand what's going on as quickly and easily as possible.
- In [Chapter 23](#), you'll learn a little about the many other varieties of outputs you can produce using R Markdown, including dashboards, websites, and books.
- We'll finish up with [Chapter 24](#), where you'll learn about the "analysis notebook" and how to systematically record your successes and failures so that you can learn from them.

Unfortunately, these chapters focus mostly on the technical mechanics of communication, not the really hard problems of communicating your thoughts to other humans. However, there are a lot of other great books about communication, which we'll point you to at the end of each chapter.

R Markdown

Introduction

R Markdown provides a unified authoring framework for data science, combining your code, its results, and your prose commentary. R Markdown documents are fully reproducible and support dozens of output formats, like PDFs, Word files, slideshows, and more.

R Markdown files are designed to be used in three ways:

- For communicating to decision makers, who want to focus on the conclusions, not the code behind the analysis.
- For collaborating with other data scientists (including future you!), who are interested in both your conclusions, and how you reached them (i.e., the code).
- As an environment in which to *do* data science, as a modern day lab notebook where you can capture not only what you did, but also what you were thinking.

R Markdown integrates a number of R packages and external tools. This means that help is, by and large, not available through ?. Instead, as you work through this chapter, and use R Markdown in the future, keep these resources close to hand:

- R Markdown Cheat Sheet: available in the RStudio IDE under *Help* → *Cheatsheets* → *R Markdown Cheat Sheet*

- R Markdown Reference Guide: available in the RStudio IDE under *Help* → *Cheatsheets* → *R Markdown Reference Guide*

Both cheatsheets are also available at <http://rstudio.com/cheatsheets>.

Prerequisites

You need the **rmarkdown** package, but you don't need to explicitly install it or load it, as RStudio automatically does both when needed.

R Markdown Basics

This is an R Markdown file, a plain-text file that has the extension *.Rmd*:

```
---
title: "Diamond sizes"
date: 2016-08-25
output: html_document
---

```{r setup, include = FALSE}
library(ggplot2)
library(dplyr)

smaller <- diamonds %>%
 filter(carat <= 2.5)
```

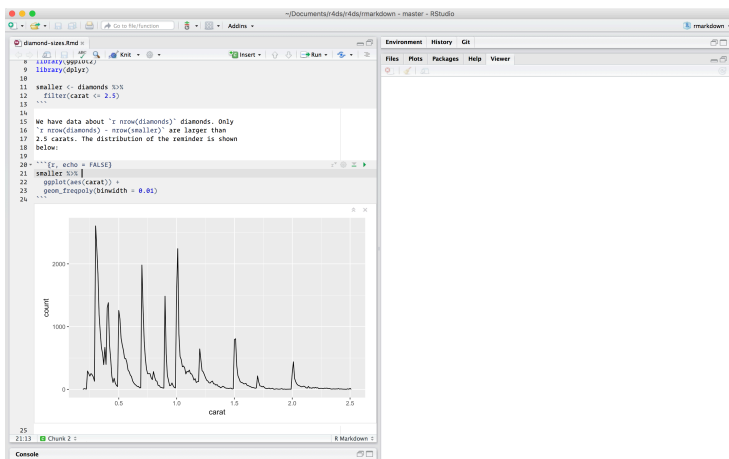
We have data about `r nrow(diamonds)` diamonds. Only
`r nrow(diamonds) - nrow(smaller)` are larger than
2.5 carats. The distribution of the remainder is shown
below:

```{r, echo = FALSE}
smaller %>%
 ggplot(aes(carat)) +
 geom_freqpoly(binwidth = 0.01)
```
```

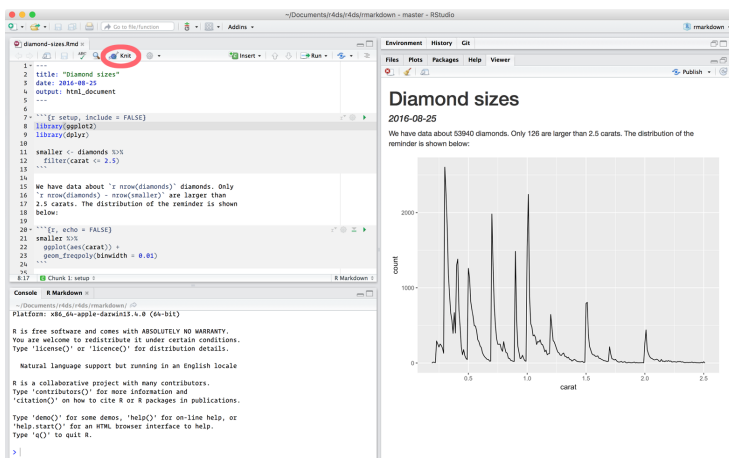
It contains three important types of content:

1. An (optional) *YAML header* surrounded by `---`s.
2. *Chunks* of R code surrounded by `````.
3. Text mixed with simple text formatting like `#` heading and `_italics_`.

When you open an *.Rmd*, you get a notebook interface where code and output are interleaved. You can run each code chunk by clicking the Run icon (it looks like a play button at the top of the chunk), or by pressing Cmd/Ctrl-Shift-Enter. RStudio executes the code and displays the results inline with the code:



To produce a complete report containing all text, code, and results, click “Knit” or press Cmd/Ctrl-Shift-K. You can also do this programmatically with `rmarkdown::render("1-example.Rmd")`. This will display the report in the viewer pane, and create a self-contained HTML file that you can share with others.



When you *knit* the document R Markdown sends the *.Rmd* file to **knitr**, which executes all of the code chunks and creates a new Markdown (*.md*) document that includes the code and its output. The Markdown file generated by **knitr** is then processed by **pandoc**, which is responsible for creating the finished file. The advantage of this two-step workflow is that you can create a very wide range of output formats, as you'll learn about in [Chapter 23](#).



To get started with your own *.Rmd* file, select *File → New File → R Markdown...* in the menu bar. RStudio will launch a wizard that you can use to pre-populate your file with useful content that reminds you how the key features of R Markdown work.

The following sections dive into the three components of an R Markdown document in more detail: the Markdown text, the code chunks, and the YAML header.

Exercises

1. Create a new notebook using *File → New File → R Notebook*. Read the instructions. Practice running the chunks. Verify that you can modify the code, rerun it, and see modified output.
2. Create a new R Markdown document with *File → New File → R Markdown...* Knit it by clicking the appropriate button. Knit it by using the appropriate keyboard shortcut. Verify that you can modify the input and see the output update.
3. Compare and contrast the R Notebook and R Markdown files you created earlier. How are the outputs similar? How are they different? How are the inputs similar? How are they different? What happens if you copy the YAML header from one to the other?
4. Create one new R Markdown document for each of the three built-in formats: HTML, PDF, and Word. Knit each of the three documents. How does the output differ? How does the input

differ? (You may need to install LaTeX in order to build the PDF output—RStudio will prompt you if this is necessary.)

Text Formatting with Markdown

Prose in *.Rmd* files is written in Markdown, a lightweight set of conventions for formatting plain-text files. Markdown is designed to be easy to read and easy to write. It is also very easy to learn. The following guide shows how to use Pandoc's Markdown, a slightly extended version of Markdown that R Markdown understands:

Text formatting

italic or *_italic_*
****bold**** **__bold__**
``code``
superscript^{2^} and subscript_{2~}

Headings

1st Level Header

2nd Level Header

3rd Level Header

Lists

* Bulleted list item 1

* Item 2

 * Item 2a

 * Item 2b

1. Numbered list item 1

1. Item 2. The numbers are incremented automatically in the output.

Links and images

<<http://example.com>>

[linked phrase](http://example.com)

![optional caption text](path/to/img.png)

Tables

```
-----
First Header	Second Header
Content Cell | Content Cell
Content Cell | Content Cell
```

The best way to learn these is simply to try them out. It will take a few days, but soon they will become second nature, and you won't need to think about them. If you forget, you can get to a handy reference sheet with *Help* → *Markdown Quick Reference*.

Exercises

1. Practice what you've learned by creating a brief CV. The title should be your name, and you should include headings for (at least) education or employment. Each of the sections should include a bulleted list of jobs/degrees. Highlight the year in bold.
2. Using the R Markdown quick reference, figure out how to:
 - a. Add a footnote.
 - b. Add a horizontal rule.
 - c. Add a block quote.
3. Copy and paste the contents of *diamond-sizes.Rmd* from <https://github.com/hadley/r4ds/tree/master/rmarkdown> into a local R Markdown document. Check that you can run it, then add text after the frequency polygon that describes its most striking features.

Code Chunks

To run code inside an R Markdown document, you need to insert a chunk. There are three ways to do so:

1. The keyboard shortcut Cmd/Ctrl-Alt-I
2. The “Insert” button icon in the editor toolbar
3. By manually typing the chunk delimiters ````\r`` and `````

Obviously, I’d recommend you learn the keyboard shortcut. It will save you a lot of time in the long run!

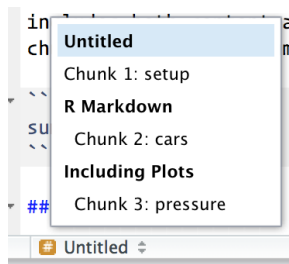
You can continue to run the code using the keyboard shortcut that by now (I hope!) you know and love: Cmd/Ctrl-Enter. However, chunks get a new keyboard shortcut: Cmd/Ctrl-Shift-Enter, which runs all the code in the chunk. Think of a chunk like a function. A chunk should be relatively self-contained, and focused around a single task.

The following sections describe the chunk header, which consists of ````\r`, followed by an optional chunk name, followed by comma-separated options, followed by `}`. Next comes your R code and the chunk end is indicated by a final `````.

Chunk Name

Chunks can be given an optional name: ````\r by-name`. This has three advantages:

- You can more easily navigate to specific chunks using the drop-down code navigator in the bottom-left of the script editor:



- Graphics produced by the chunks will have useful names that make them easier to use elsewhere. More on that in “[Other Important Options](#)” on page 467.

- You can set up networks of cached chunks to avoid re-performing expensive computations on every run. More on that in a bit.

There is one chunk name that imbues special behavior: `setup`. When you're in a notebook mode, the chunk named `setup` will be run automatically once, before any other code is run.

Chunk Options

Chunk output can be customized with *options*, arguments supplied to the chunk header. **knitr** provides almost 60 options that you can use to customize your code chunks. Here we'll cover the most important chunk options that you'll use frequently. You can see the full list at <http://yihui.name/knitr/options/>.

The most important set of options controls if your code block is executed and what results are inserted in the finished report:

- `eval = FALSE` prevents code from being evaluated. (And obviously if the code is not run, no results will be generated.) This is useful for displaying example code, or for disabling a large block of code without commenting each line.
- `include = FALSE` runs the code, but doesn't show the code or results in the final document. Use this for setup code that you don't want cluttering your report.
- `echo = FALSE` prevents code, but not the results from appearing in the finished file. Use this when writing reports aimed at people who don't want to see the underlying R code.
- `message = FALSE` or `warning = FALSE` prevents messages or warnings from appearing in the finished file.
- `results = 'hide'` hides printed output; `fig.show = 'hide'` hides plots.
- `error = TRUE` causes the render to continue even if code returns an error. This is rarely something you'll want to include in the final version of your report, but can be very useful if you need to debug exactly what is going on inside your *.Rmd*. It's also useful if you're teaching R and want to deliberately include an error. The default, `error = FALSE`, causes knitting to fail if there is a single error in the document.

The following table summarizes which types of output each option suppresses:

| Option | Run code | Show code | Output | Plots | Messages | Warnings |
|-------------------|----------|-----------|--------|-------|----------|----------|
| eval = FALSE | X | | X | X | X | X |
| include = FALSE | | X | X | X | X | X |
| echo = FALSE | | X | | | | |
| results = "hide" | | | X | | | |
| fig.show = "hide" | | | | X | | |
| message = FALSE | | | | | X | |
| warning = FALSE | | | | | | X |

Table

By default, R Markdown prints data frames and matrices as you'd see them in the console:

```
mtcars[1:5, 1:10]
#>      mpg cyl disp  hp drat   wt  qsec vs am gear
#> Mazda RX4      21.0   6  160  110  3.90  2.62 16.5  0  1    4
#> Mazda RX4 Wag  21.0   6  160  110  3.90  2.88 17.0  0  1    4
#> Datsun 710     22.8   4  108   93  3.85  2.32 18.6  1  1    4
#> Hornet 4 Drive  21.4   6  258  110  3.08  3.21 19.4  1  0    3
#> Hornet Sportabout 18.7   8  360  175  3.15  3.44 17.0  0  0    3
```

If you prefer that data be displayed with additional formatting you can use the `knitr::kable` function. The following code generates [Table 21-1](#):

```
knitr::kable(
  mtcars[1:5, ],
  caption = "A knitr kable."
)
```

Table 21-1. A knitr kable

| | mpg | cyl | disp | hp | drat | wt | qsec | vs | am | gear | carb |
|-------------------|------|-----|------|-----|------|------|------|----|----|------|------|
| Mazda RX4 | 21.0 | 6 | 160 | 110 | 3.90 | 2.62 | 16.5 | 0 | 1 | 4 | 4 |
| Mazda RX4 Wag | 21.0 | 6 | 160 | 110 | 3.90 | 2.88 | 17.0 | 0 | 1 | 4 | 4 |
| Datsun 710 | 22.8 | 4 | 108 | 93 | 3.85 | 2.32 | 18.6 | 1 | 1 | 4 | 1 |
| Hornet 4 Drive | 21.4 | 6 | 258 | 110 | 3.08 | 3.21 | 19.4 | 1 | 0 | 3 | 1 |
| Hornet Sportabout | 18.7 | 8 | 360 | 175 | 3.15 | 3.44 | 17.0 | 0 | 0 | 3 | 2 |

Read the documentation for `?knitr::kable` to see the other ways in which you can customize the table. For even deeper customization, consider the **xtable**, **stargazer**, **pander**, **tables**, and **ascii** packages. Each provides a set of tools for returning formatted tables from R code.

There is also a rich set of options for controlling how figures are embedded. You'll learn about these in [“Saving Your Plots” on page 464](#).

Caching

Normally, each knit of a document starts from a completely clean slate. This is great for reproducibility, because it ensures that you've captured every important computation in code. However, it can be painful if you have some computations that take a long time. The solution is `cache = TRUE`. When set, this will save the output of the chunk to a specially named file on disk. On subsequent runs, **knitr** will check to see if the code has changed, and if it hasn't, it will reuse the cached results.

The caching system must be used with care, because by default it is based on the code only, not its dependencies. For example, here the `processed_data` chunk depends on the `raw_data` chunk:

```
```{r raw_data}
rawdata <- readr::read_csv("a_very_large_file.csv")
...

```{r processed_data, cached = TRUE}
processed_data <- rawdata %>%
  filter(!is.na(import_var)) %>%
  mutate(new_variable = complicated_transformation(x, y, z))
...`
```

Caching the `processed_data` chunk means that it will get rerun if the **dplyr** pipeline is changed, but it won't get rerun if the `read_csv()` call changes. You can avoid that problem with the `dependson` chunk option:

```
```{r processed_data, cached = TRUE, dependson = "raw_data"}
processed_data <- rawdata %>%
 filter(!is.na(import_var)) %>%
 mutate(new_variable = complicated_transformation(x, y, z))
...`
```

dependson should contain a character vector of *every* chunk that the cached chunk depends on. **knitr** will update the results for the cached chunk whenever it detects that one of its dependencies has changed.

Note that the chunks won't update if *a\_very\_large\_file.csv* changes, because **knitr** caching only tracks changes within the *.Rmd* file. If you want to also track changes to that file you can use the `cache.extra` option. This is an arbitrary R expression that will invalidate the cache whenever it changes. A good function to use is `file.info()`: it returns a bunch of information about the file including when it was last modified. Then you can write:

```
```{r raw_data, cache.extra = file.info("a_very_large_file.csv")}
rawdata <- readr::read_csv("a_very_large_file.csv")
```
```

As your caching strategies get progressively more complicated, it's a good idea to regularly clear out all your caches with `knitr::clean_cache()`.

I've used the advice of [David Robinson](#) to name these chunks: each chunk is named after the primary object that it creates. This makes it easier to understand the dependson specification.

## Global Options

As you work more with **knitr**, you will discover that some of the default chunk options don't fit your needs, and want to change them. You can do that by calling `knitr::opts_chunk$set()` in a code chunk. For example, when writing books and tutorials I set:

```
knitr::opts_chunk$set(
 comment = "#>",
 collapse = TRUE
)
```

This uses my preferred comment formatting, and ensures that the code and output are kept closely entwined. On the other hand, if you were preparing a report, you might set:

```
knitr::opts_chunk$set(
 echo = FALSE
)
```

That will hide the code by default, only showing the chunks you deliberately choose to show (with `echo = TRUE`). You might con-

sider setting `message = FALSE` and `warning = FALSE`, but that would make it harder to debug problems because you wouldn't see any messages in the final document.

## Inline Code

There is one other way to embed R code into an R Markdown document: directly into the text, with: ``r ``. This can be very useful if you mention properties of your data in the text. For example, in the example document I used at the start of the chapter I had:

```
We have data about `r nrow(diamonds)` diamonds. Only `r
nrow(diamonds) - nrow(smaller)` are larger than 2.5 carats. The
distribution of the remainder is shown below:
```

When the report is knitted, the results of these computations are inserted into the text:

```
We have data about 53940 diamonds. Only 126 are larger than 2.5
carats. The distribution of the remainder is shown below:
```

When inserting numbers into text, `format()` is your friend. It allows you to set the number of digits so you don't print to a ridiculous degree of accuracy, and a `big.mark` to make numbers easier to read. I'll often combine these into a helper function:

```
comma <- function(x) format(x, digits = 2, big.mark = ",")
comma(3452345)
#> [1] "3,452,345"
comma(.12358124331)
#> [1] "0.12"
```

## Exercises

1. Add a section that explores how diamond sizes vary by cut, color, and clarity. Assume you're writing a report for someone who doesn't know R, and instead of setting `echo = FALSE` on each chunk, set a global option.
2. Download *diamond-sizes.Rmd* from <https://github.com/hadley/r4ds/tree/master/rmarkdown>. Add a section that describes the largest 20 diamonds, including a table that displays their most important attributes.
3. Modify *diamonds-sizes.Rmd* to use `comma()` to produce nicely formatted output. Also include the percentage of diamonds that are larger than 2.5 carats.



4. Set up a network of chunks where `d` depends on `c` and `b`, and both `b` and `c` depend on `a`. Have each chunk print `lubridate::now()`, set `cache = TRUE`, then verify your understanding of caching.

## Troubleshooting

Troubleshooting R Markdown documents can be challenging because you are no longer in an interactive R environment, and you will need to learn some new tricks. The first thing you should always try is to re-create the problem in an interactive session. Restart R, then “Run all chunks” (either from the the Code menu, under the Run region, or with the keyboard shortcut Ctrl-Alt-R). If you’re lucky, that will re-create the problem, and you can figure out what’s going on interactively.

If that doesn’t help, there must be something different between your interactive environment and the R Markdown environment. You’re going to need to systematically explore the options. The most common difference is the working directory: the working directory of an R Markdown document is the directory in which it lives. Check that the working directory is what you expect by including `getwd()` in a chunk.

Next, brainstorm all of the things that might cause the bug. You’ll need to systematically check that they’re the same in your R session and your R Markdown session. The easiest way to do that is to set `error = TRUE` on the chunk causing the problem, then use `print()` and `str()` to check that settings are as you expect.

## YAML Header

You can control many other “whole document” settings by tweaking the parameters of the YAML header. You might wonder what YAML stands for: it’s “yet another markup language,” which is designed for representing hierarchical data in a way that’s easy for humans to read and write. R Markdown uses it to control many details of the output. Here we’ll discuss two: document parameters and bibliographies.

## Parameters

R Markdown documents can include one or more parameters whose values can be set when you render the report. Parameters are useful when you want to re-render the same report with distinct values for various key inputs. For example, you might be producing sales reports per branch, exam results by student, or demographic summaries by country. To declare one or more parameters, use the `params` field.

This example uses a `my_class` parameter to determine which class of cars to display:

```

output: html_document
params:
 my_class: "suv"

```{r setup, include = FALSE}
library(ggplot2)
library(dplyr)

class <- mpg %>% filter(class == params$my_class)
```

Fuel economy for `r params$my_class`s

```{r, message = FALSE}
ggplot(class, aes(displ, hwy)) +
  geom_point() +
  geom_smooth(se = FALSE)
```
```

As you can see, parameters are available within the code chunks as a read-only list named `params`.

You can write atomic vectors directly into the YAML header. You can also run arbitrary R expressions by prefacing the parameter value with `!r`. This is a good way to specify date/time parameters:

```
params:
 start: !r lubridate::ymd("2015-01-01")
 snapshot: !r lubridate::ymd_hms("2015-01-01 12:30:00")
```

In RStudio, you can click the “Knit with Parameters” option in the Knit drop-down menu to set parameters, render, and preview the report in a single user-friendly step. You can customize the dialog by

setting other options in the header. See <http://bit.ly/ParamReports> for more details.

Alternatively, if you need to produce many such parameterized reports, you can call `rmarkdown::render()` with a list of params:

```
rmarkdown::render(
 "fuel-economy.Rmd",
 params = list(my_class = "suv")
)
```

This is particularly powerful in conjunction with `purrr::pwalk()`. The following example creates a report for each value of `class` found in `mpg`. First we create a data frame that has one row for each class, giving the filename of report and the params it should be given:

```
reports <- tibble(
 class = unique(mpg$class),
 filename = stringr::str_c("fuel-economy-", class, ".html"),
 params = purrr::map(class, ~ list(my_class = .))
)
reports
#> # A tibble: 7 × 3
#> class filename params
#> <chr> <chr> <list>
#> 1 compact fuel-economy-compact.html <list [1]>
#> 2 midsize fuel-economy-midsize.html <list [1]>
#> 3 suv fuel-economy-suv.html <list [1]>
#> 4 2seater fuel-economy-2seater.html <list [1]>
#> 5 minivan fuel-economy-minivan.html <list [1]>
#> 6 pickup fuel-economy-pickup.html <list [1]>
#> # ... with 1 more rows
```

Then we match the column names to the argument names of `render()`, and use **purrr**'s *parallel* walk to call `render()` once for each row:

```
reports %>%
 select(output_file = filename, params) %>%
 purrr::pwalk(rmarkdown::render, input = "fuel-economy.Rmd")
```

## Bibliographies and Citations

Pandoc can automatically generate citations and a bibliography in a number of styles. To use this feature, specify a bibliography file using the `bibliography` field in your file's header. The field should contain a path from the directory that contains your `.Rmd` file to the file that contains the bibliography file:

```
bibliography: rmarkdown.bib
```

You can use many common bibliography formats including BibLaTeX, BibTeX, endnote, and medline.

To create a citation within your *.Rmd* file, use a key composed of “@” and the citation identifier from the bibliography file. Then place the citation in square brackets. Here are some examples:

```
Separate multiple citations with a `;`:
Blah blah [@smith04; @doe99].
```

```
You can add arbitrary comments inside the square brackets:
Blah blah [see @doe99, pp. 33-35; also @smith04, ch. 1].
```

```
Remove the square brackets to create an in-text citation:
@smith04 says blah, or @smith04 [p. 33] says blah.
```

```
Add a `-' before the citation to suppress the author's name:
Smith says blah [-@smith04].
```

When R Markdown renders your file, it will build and append a bibliography to the end of your document. The bibliography will contain each of the cited references from your bibliography file, but it will not contain a section heading. As a result it is common practice to end your file with a section header for the bibliography, such as # References or # Bibliography.

You can change the style of your citations and bibliography by referencing a CSL (citation style language) file to the `cs1` field:

```
bibliography: rmarkdown.bib
cs1: apa.csl
```

As with the bibliography field, your CSL file should contain a path to the file. Here I assume that the CSL file is in the same directory as the *.Rmd* file. A good place to find CSL style files for common bibliography styles is <http://github.com/citation-style-language/styles>.

## Learning More

R Markdown is still relatively young, and is still growing rapidly. The best place to stay on top of innovations is the official R Markdown website: <http://rmarkdown.rstudio.com>.

There are two important topics that we haven't covered here: collaboration, and the details of accurately communicating your ideas to other humans. Collaboration is a vital part of modern data science,

and you can make your life much easier by using version control tools, like Git and GitHub. We recommend two free resources that will teach you about Git:

- “Happy Git with R”: a user-friendly introduction to Git and GitHub from R users, by Jenny Bryan. The book is [freely available online](#).
- The “Git and GitHub” chapter of *R Packages*, by Hadley. You can also read it for free online: <http://r-pkgs.had.co.nz/git.html>.

I have also not talked about what you should actually write in order to clearly communicate the results of your analysis. To improve your writing, I highly recommend reading either *Style: Lessons in Clarity and Grace* by Joseph M. Williams and Joseph Bizup, or *The Sense of Structure: Writing from the Reader’s Perspective* by George Gopen. Both books will help you understand the structure of sentences and paragraphs, and give you the tools to make your writing more clear. (These books are rather expensive if purchased new, but they’re used by many English classes so there are plenty of cheap secondhand copies). George Gopen also has a number of [short articles on writing](#). They are aimed at lawyers, but almost everything applies to data scientists too.



---

# Graphics for Communication with ggplot2

## Introduction

In [Chapter 5](#), you learned how to use plots as tools for *exploration*. When you make exploratory plots, you know—even before looking—which variables the plot will display. You made each plot for a purpose, could quickly look at it, and then move on to the next plot. In the course of most analyses, you’ll produce tens or hundreds of plots, most of which are immediately thrown away.

Now that you understand your data, you need to *communicate* your understanding to others. Your audience will likely not share your background knowledge and will not be deeply invested in the data. To help others quickly build up a good mental model of the data, you will need to invest considerable effort in making your plots as self-explanatory as possible. In this chapter, you’ll learn some of the tools that **ggplot2** provides to do so.

This chapter focuses on the tools you need to create good graphics. I assume that you know what you want, and just need to know how to do it. For that reason, I highly recommend pairing this chapter with a good general visualization book. I particularly like *The Truthful Art*, by Albert Cairo. It doesn’t teach the mechanics of creating visualizations, but instead focuses on what you need to think about in order to create effective graphics.

## Prerequisites

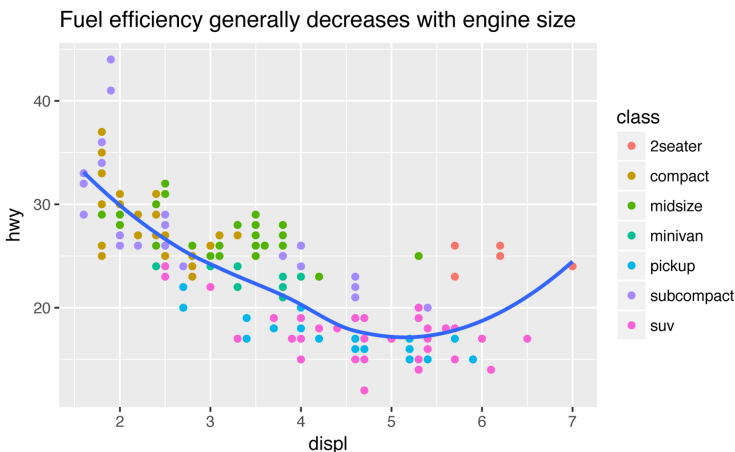
In this chapter, we'll focus once again on **ggplot2**. We'll also use a little **dplyr** for data manipulation, and a few **ggplot2** extension packages, including **ggrepel** and **viridis**. Rather than loading those extensions here, we'll refer to their functions explicitly, using the `::` notation. This will help make it clear which functions are built into **ggplot2**, and which come from other packages. Don't forget you'll need to install those packages with `install.packages()` if you don't already have them.

```
library(tidyverse)
```

## Label

The easiest place to start when turning an exploratory graphic into an expository graphic is with good labels. You add labels with the `labs()` function. This example adds a plot title:

```
ggplot(mpg, aes(displ, hwy)) +
 geom_point(aes(color = class)) +
 geom_smooth(se = FALSE) +
 labs(
 title = paste(
 "Fuel efficiency generally decreases with"
 "engine size"
)
)
```



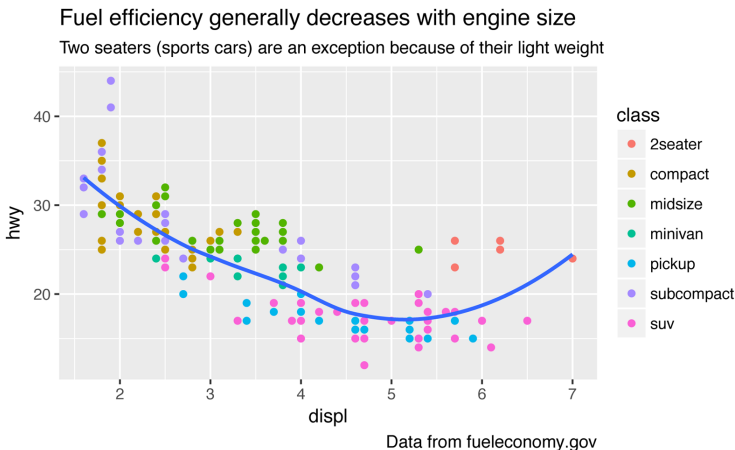


The purpose of a plot title is to summarize the main finding. Avoid titles that just describe what the plot is, e.g., “A scatterplot of engine displacement vs. fuel economy.”

If you need to add more text, there are two other useful labels that you can use in **ggplot2** 2.2.0 and above (which should be available by the time you’re reading this book):

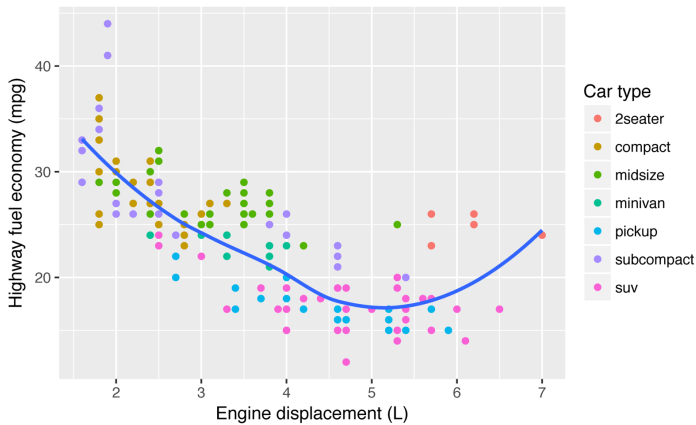
- `subtitle` adds additional detail in a smaller font beneath the title.
- `caption` adds text at the bottom right of the plot, often used to describe the source of the data:

```
ggplot(mpg, aes(displ, hwy)) +
 geom_point(aes(color = class)) +
 geom_smooth(se = FALSE) +
 labs(
 title = paste(
 "Fuel efficiency generally decreases with"
 "engine size",
),
 subtitle = paste(
 "Two seaters (sports cars) are an exception"
 "because of their light weight",
),
 caption = "Data from fueleconomy.gov"
)
```



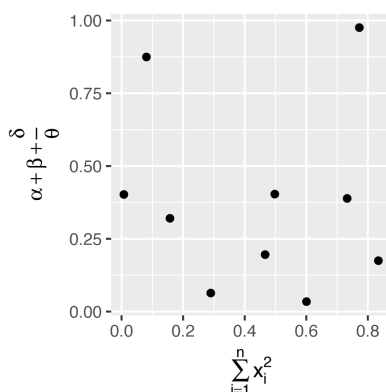
You can also use `labs()` to replace the axis and legend titles. It's usually a good idea to replace short variable names with more detailed descriptions, and to include the units:

```
ggplot(mpg, aes(displ, hwy)) +
 geom_point(aes(color = class)) +
 geom_smooth(se = FALSE) +
 labs(
 x = "Engine displacement (L)",
 y = "Highway fuel economy (mpg)",
 colour = "Car type"
)
```



It's possible to use mathematical equations instead of text strings. Just switch `"` out for `quote()` and read about the available options in `?plotmath`:

```
df <- tibble(
 x = runif(10),
 y = runif(10)
)
ggplot(df, aes(x, y)) +
 geom_point() +
 labs(
 x = quote(sum(x[i] ^ 2, i == 1, n)),
 y = quote(alpha + beta + frac(delta, theta))
)
```



## Exercises

1. Create one plot on the fuel economy data with customized title, subtitle, caption, x, y, and colour labels.
2. The `geom_smooth()` is somewhat misleading because the hwy for large engines is skewed upwards due to the inclusion of light-weight sports cars with big engines. Use your modeling tools to fit and display a better model.
3. Take an exploratory graphic that you've created in the last month, and add informative titles to make it easier for others to understand.

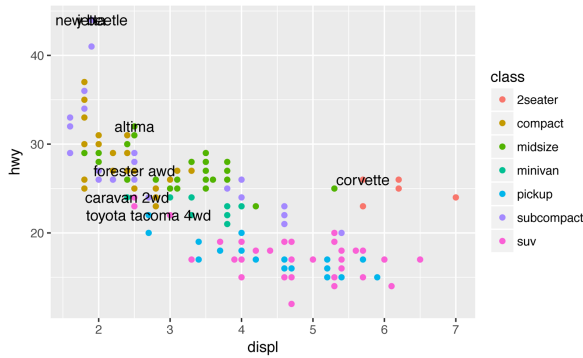
## Annotations

In addition to labeling major components of your plot, it's often useful to label individual observations or groups of observations. The first tool you have at your disposal is `geom_text()`. `geom_text()` is similar to `geom_point()`, but it has an additional aesthetic: `label`. This makes it possible to add textual labels to your plots.

There are two possible sources of labels. First, you might have a tibble that provides labels. The following plot isn't terribly useful, but it illustrates a useful approach—pull out the most efficient car in each class with `dplyr`, and then label it on the plot:

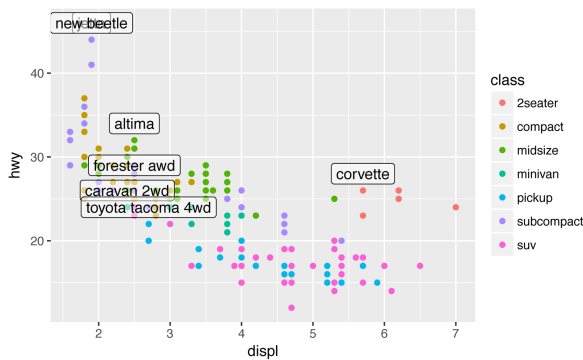
```
best_in_class <- mpg %>%
 group_by(class) %>%
 filter(row_number(desc(hwy)) == 1)
```

```
ggplot(mpg, aes(displ, hwy)) +
 geom_point(aes(color = class)) +
 geom_text(aes(label = model), data = best_in_class)
```



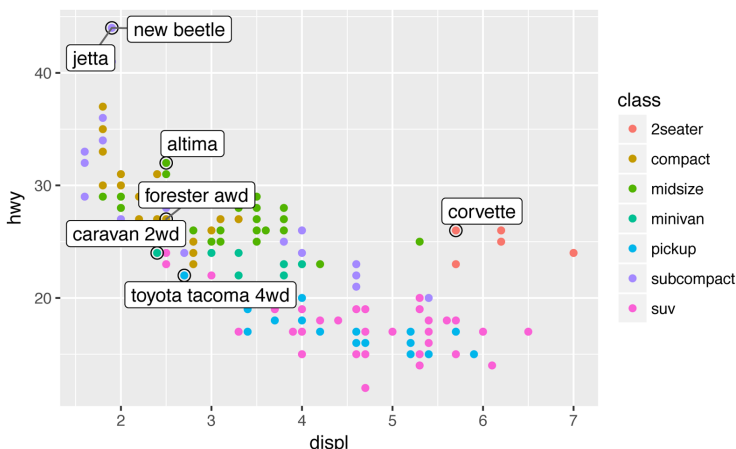
This is hard to read because the labels overlap with each other, and with the points. We can make things a little better by switching to `geom_label()`, which draws a rectangle behind the text. We also use the `nudge_y` parameter to move the labels slightly above the corresponding points:

```
ggplot(mpg, aes(displ, hwy)) +
 geom_point(aes(color = class)) +
 geom_label(
 aes(label = model),
 data = best_in_class,
 nudge_y = 2,
 alpha = 0.5
)
```



That helps a bit, but if you look closely in the top lefthand corner, you'll notice that there are two labels practically on top of each other. This happens because the highway mileage and displacement for the best cars in the compact and subcompact categories are exactly the same. There's no way that we can fix these by applying the same transformation for every label. Instead, we can use the **ggrepel** package by Kamil Slowikowski. This useful package will automatically adjust labels so that they don't overlap:

```
ggplot(mpg, aes(displ, hwy)) +
 geom_point(aes(color = class)) +
 geom_point(size = 3, shape = 1, data = best_in_class) +
 ggrepel::geom_label_repel(
 aes(label = model),
 data = best_in_class
)
```

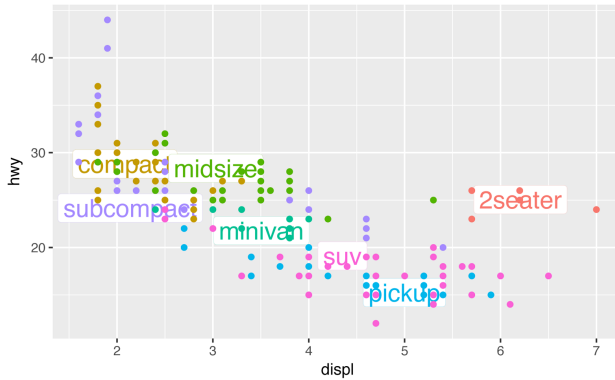


Note another handy technique used here: I added a second layer of large, hollow points to highlight the points that I've labeled.

You can sometimes use the same idea to replace the legend with labels placed directly on the plot. It's not wonderful for this plot, but it isn't too bad. (`theme(legend.position = "none")` turns the legend off—we'll talk about it more shortly.)

```
class_avg <- mpg %>%
 group_by(class) %>%
 summarize(
 displ = median(displ),
 hwy = median(hwy)
)
```

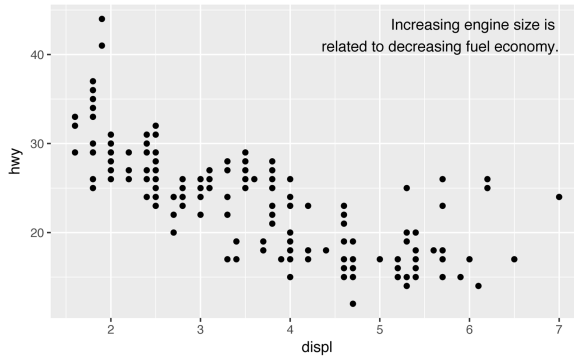
```
ggplot(mpg, aes(displ, hwy, color = class)) +
 ggrepel::geom_label_repel(aes(label = class),
 data = class_avg,
 size = 6,
 label.size = 0,
 segment.color = NA
) +
 geom_point() +
 theme(legend.position = "none")
```



Alternatively, you might just want to add a single label to the plot, but you'll still need to create a data frame. Often, you want the label in the corner of the plot, so it's convenient to create a new data frame using `summarize()` to compute the maximum values of `x` and `y`:

```
label <- mpg %>%
 summarize(
 displ = max(displ),
 hwy = max(hwy),
 label = paste(
 "Increasing engine size is \nrelated to"
 "decreasing fuel economy."
)
)

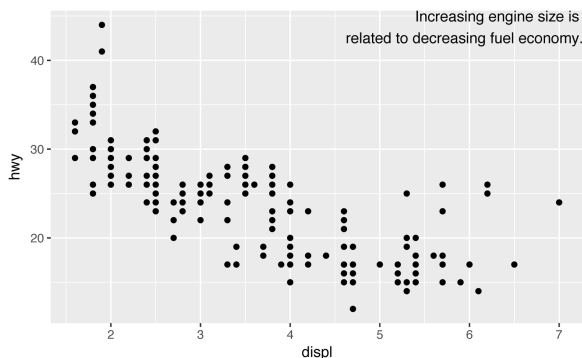
ggplot(mpg, aes(displ, hwy)) +
 geom_point() +
 geom_text(
 aes(label = label),
 data = label,
 vjust = "top",
 hjust = "right"
)
```



If you want to place the text exactly on the borders of the plot, you can use `+Inf` and `-Inf`. Since we're no longer computing the positions from `mpg`, we can use `tibble()` to create the data frame:

```
label <- tibble(
 displ = Inf,
 hwy = Inf,
 label = paste(
 "Increasing engine size is \nrelated to"
 "decreasing fuel economy."
)
)

ggplot(mpg, aes(displ, hwy)) +
 geom_point() +
 geom_text(
 aes(label = label),
 data = label,
 vjust = "top",
 hjust = "right"
)
```



In these examples, I manually broke the label up into lines using "\n". Another approach is to use `stringr::str_wrap()` to automatically add line breaks, given the number of characters you want per line:

```
"Increasing engine size related to decreasing fuel economy." %>%
 stringr::str_wrap(width = 40) %>%
 writelines()
#> Increasing engine size is related to
#> decreasing fuel economy.
```

Note the use of `hjust` and `vjust` to control the alignment of the label. **Figure 22-1** shows all nine possible combinations.

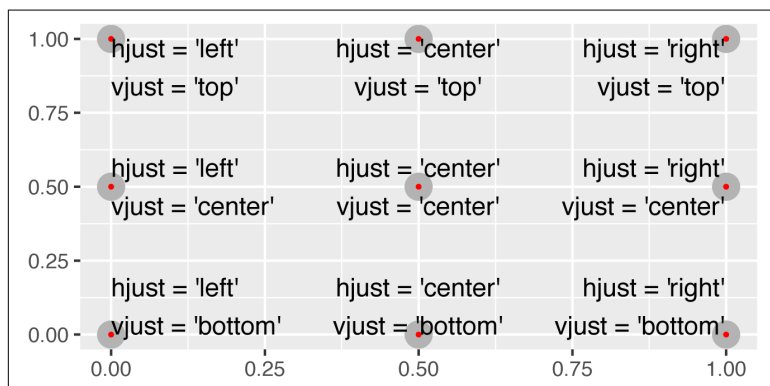


Figure 22-1. All nine combinations of `hjust` and `vjust`

Remember, in addition to `geom_text()`, you have many other geoms in **ggplot2** available to help annotate your plot. A few ideas:

- Use `geom_hline()` and `geom_vline()` to add reference lines. I often make them thick (`size = 2`) and white (`color = white`), and draw them underneath the primary data layer. That makes them easy to see, without drawing attention away from the data.
- Use `geom_rect()` to draw a rectangle around points of interest. The boundaries of the rectangle are defined by the `xmin`, `xmax`, `ymin`, and `ymax` aesthetics.
- Use `geom_segment()` with the `arrow` argument to draw attention to a point with an arrow. Use the `x` and `y` aesthetics to define the starting location, and `xend` and `yend` to define the end location.



The only limit is your imagination (and your patience with positioning annotations to be aesthetically pleasing)!

## Exercises

1. Use `geom_text()` with infinite positions to place text at the four corners of the plot.
2. Read the documentation for `annotate()`. How can you use it to add a text label to a plot without having to create a tibble?
3. How do labels with `geom_text()` interact with faceting? How can you add a label to a single facet? How can you put a different label in each facet? (Hint: think about the underlying data.)
4. What arguments to `geom_label()` control the appearance of the background box?
5. What are the four arguments to `arrow()`? How do they work? Create a series of plots that demonstrate the most important options.

## Scales

The third way you can make your plot better for communication is to adjust the scales. Scales control the mapping from data values to things that you can perceive. Normally, **ggplot2** automatically adds scales for you. For example, when you type:

```
ggplot(mpg, aes(displ, hwy)) +
 geom_point(aes(color = class))
```

**ggplot2** automatically adds default scales behind the scenes:

```
ggplot(mpg, aes(displ, hwy)) +
 geom_point(aes(color = class)) +
 scale_x_continuous() +
 scale_y_continuous() +
 scale_color_discrete()
```

Note the naming scheme for scales: `scale_` followed by the name of the aesthetic, then `_`, then the name of the scale. The default scales are named according to the type of variable they align with: continuous, discrete, datetime, or date. There are lots of nondefault scales, which you'll learn about next.

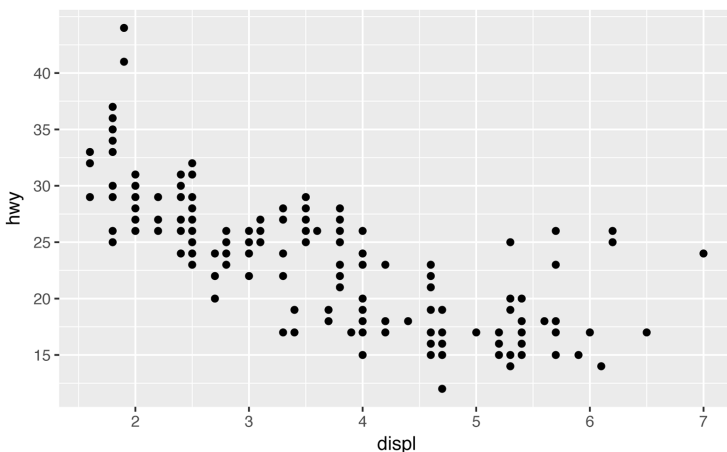
The default scales have been carefully chosen to do a good job for a wide range of inputs. Nevertheless, you might want to override the defaults for two reasons:

- You might want to tweak some of the parameters of the default scale. This allows you to do things like change the breaks on the axes, or the key labels on the legend.
- You might want to replace the scale altogether, and use a completely different algorithm. Often you can do better than the default because you know more about the data.

## Axis Ticks and Legend Keys

There are two primary arguments that affect the appearance of the ticks on the axes and the keys on the legend: `breaks` and `labels`. `breaks` controls the position of the ticks, or the values associated with the keys. `labels` controls the text label associated with each tick/key. The most common use of `breaks` is to override the default choice:

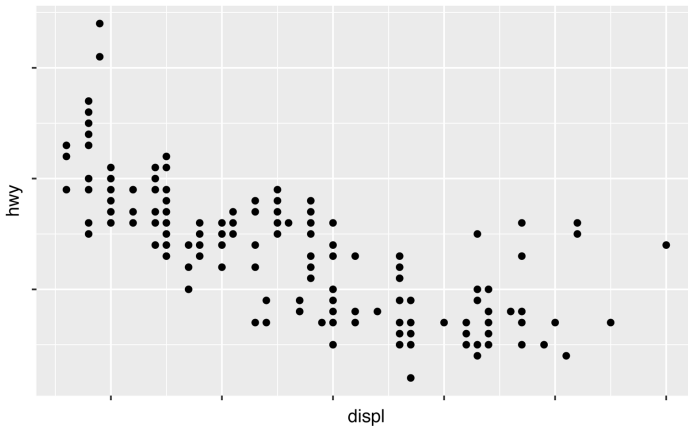
```
ggplot(mpg, aes(displ, hwy)) +
 geom_point() +
 scale_y_continuous(breaks = seq(15, 40, by = 5))
```



You can use `labels` in the same way (a character vector the same length as `breaks`), but you can also set it to `NULL` to suppress the

labels altogether. This is useful for maps, or for publishing plots where you can't share the absolute numbers:

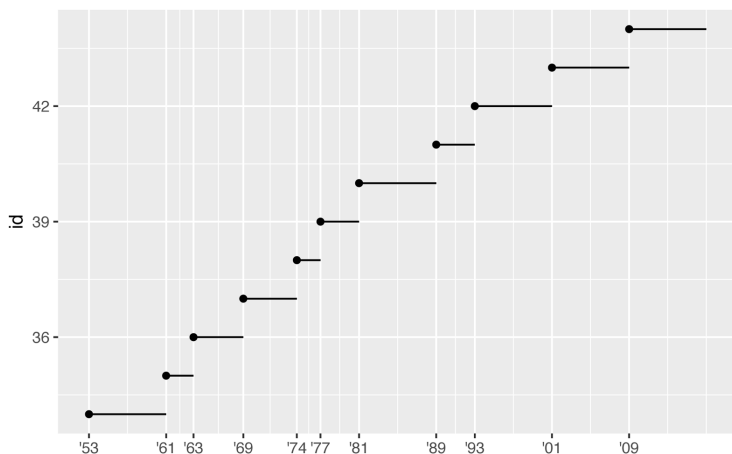
```
ggplot(mpg, aes(displ, hwy)) +
 geom_point() +
 scale_x_continuous(labels = NULL) +
 scale_y_continuous(labels = NULL)
```



You can also use `breaks` and `labels` to control the appearance of legends. Collectively axes and legends are called *guides*. Axes are used for the x and y aesthetics; legends are used for everything else.

Another use of `breaks` is when you have relatively few data points and want to highlight exactly where the observations occur. For example, take this plot that shows when each US president started and ended their term:

```
presidential %>%
 mutate(id = 33 + row_number()) %>%
 ggplot(aes(start, id)) +
 geom_point() +
 geom_segment(aes(xend = end, yend = id)) +
 scale_x_date(
 NULL,
 breaks = presidential$start,
 date_labels = "%y"
)
```



Note that the specification of breaks and labels for date and date-time scales is a little different:

- `date_labels` takes a format specification, in the same form as `parse_datetime()`.
- `date_breaks` (not shown here) takes a string like “2 days” or “1 month”.

## Legend Layout

You will most often use `breaks` and `labels` to tweak the axes. While they both also work for legends, there are a few other techniques you are more likely to use.

To control the overall position of the legend, you need to use a `theme()` setting. We’ll come back to themes at the end of the chapter, but in brief, they control the nondata parts of the plot. The theme setting `legend.position` controls where the legend is drawn:

```
base <- ggplot(mpg, aes(displ, hwy)) +
 geom_point(aes(color = class))

base + theme(legend.position = "left")
base + theme(legend.position = "top")
base + theme(legend.position = "bottom")
base + theme(legend.position = "right") # the default
```